2.Introduction to Python

Q1:What is Program?

Ans: An ordered set of instructions or commands to be executed by a computer is called a program

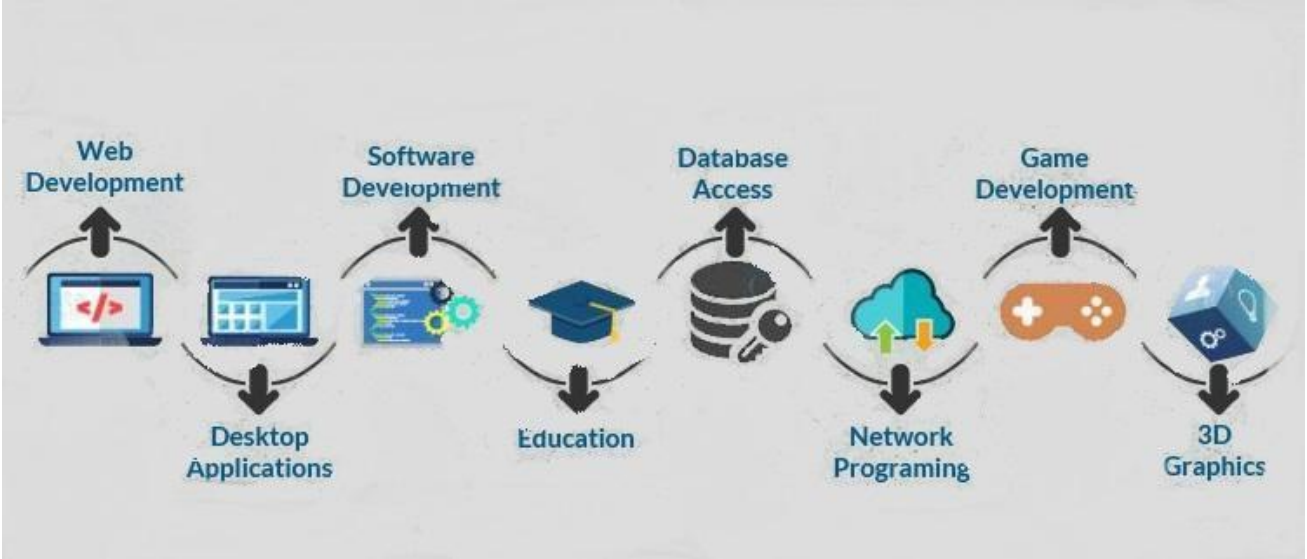
Q2:What is Programming Language?

Ans : The language used to specify set of instructions to the computer is called a programming language .Example : Python, C, C++, Java, etc.

Q3.What is Python?

Ans:Python is a programming,highlevel,interpreted,portable,free&open source language, as Python programs are executed by an interpreter. Created by **Guido van Rossum** in February,1991.Python got its name from a BBC comedy series from seventies called "**Monty Pythons Flying Circus**". It is based on two programming languages called **ABC and Modula–3**.

Q4: What are the Advantages of Python?



1. **Easy to Use: Python is compact and very easy to use with very simple syntax rules. It is programmer–friendly**
2. **Interpreted Language: Python interprets and executes the code line by line at a time. It makes Python an easy–to–debug language and thus suitable for beginners and advanced users.programs written in Python are easily readable and understandable**
3. **Expressive Language: Because of simple syntax and fewer lines of code, it is more capable to express code's purpose than many other languages**
4. **Completeness: Python Standard Library provides various modules for different functionalities.**

5.Cross–Platform Language: Python can run on different platforms like Windows, Linux / Unix, Macintosh, Super Computers, Smart Phones etc. Hence it portable language

6.Free and Open Source: Python is freely available at free of cost and its source code is available to every body for further improvement

7.Variety of Usage: Python can be used for a variety of applications like Software development, Scripting, Web Applications, ,Game development, System Administration, Rapid Prototyping, GUI Programs, Database Applications etc., scientific computing, big data and Artificial Intelligence.

8.Case Sensitive:It is case sensitive. i.e. Uppercase and Lowercase alphabets are different

3.1.1 Working with Python

To write and run Python program, install Python interpreter in computer.

IDLE (GUI integrated) is the standard, most popular Python development environment.

Python shell(interpreter) can be used in 2 ways **(i)Interactive Mode**

(ii) Script Mode

Other offline IDE's are: **1.** Visual Studio Code 2.PyCharm, 3. Jupyter Note Book
4. Anaconda Navigator 5.Spyder 6.Thonny 7.IDLE
8.Sublime Text 9.Wing IDE 10. ATOM

Beginner / Education: Thonny, IDLE

Data Science: Jupyter, Spyder, Anaconda

Professional Development: PyCharm, VS Code

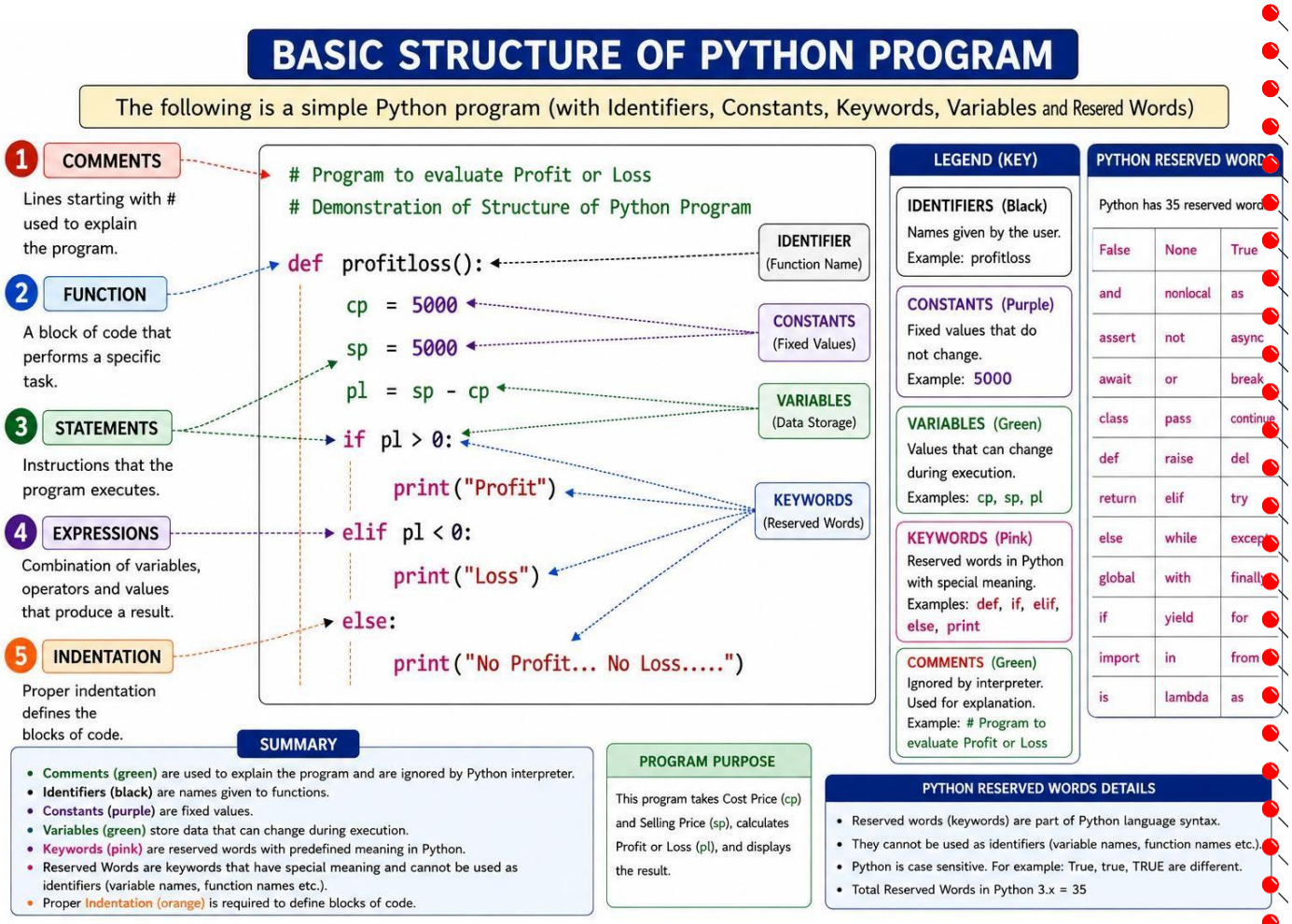
Lightweight Coding: Sublime Text

Other online IDE's are: 1. Google Colab 2.Replit 3. PythonAnywhere
4. Jupyter Notebook (Binder) 5. OneCompiler
6.Programiz 7.OnlineGDB 8. JDoodle
9.Pythontutor 10.Trinket

(i) Interactive Mode: As the name suggests, this mode allows to interact with OS. This mode does not save commands in form of a program. The symbol **'>>>'** is called Python prompt, indicates that interpreter is ready to accept command.

(ii) Script Mode: In the script mode, we can write a Python program in a file, save it and then use the interpreter to execute the program from the file. Such program files have a **.py** extension and they are also known as scripts.

Basic Structure Of Python Program:



1.Comments: These are the statements that will be ignored by Python interpreter .A single line comment starts with the **symbol #.** For multiline comment content will be enclosed in triple quotes (" " ") or triple apostrophe (' ' ').

2.Functions: A function is a code that has collection of statements to do some task. It has a name and a function can be executed repeatedly as many times required

def wish(name): **print("HAPPY BIRTH DAY Mr.", name)** **wish("Syed Rafiq")**	**def rip(name):** **print("REST IN PEACE Late",name)** **rip("Syed Rafiq")**

```
def rip(name):
    print("REST IN PEACE Late", name)
rip("Syed Rafiq")
def  wish(name):
        print("HAPPY BIRTH DAY Mr.", name)
wish("Syed Rafiq")
```

3.Statement:A statement is a programming instruction that does something i.e. performs some action.

Ex:x=10,age=15
name = input("Enter your name: ")
print("Welcome", name)

4. Expression: An expression in Python is a combination of values,literals, operators and operands that is evaluated to produce some other values An expression is anything that gives a value after evaluation.

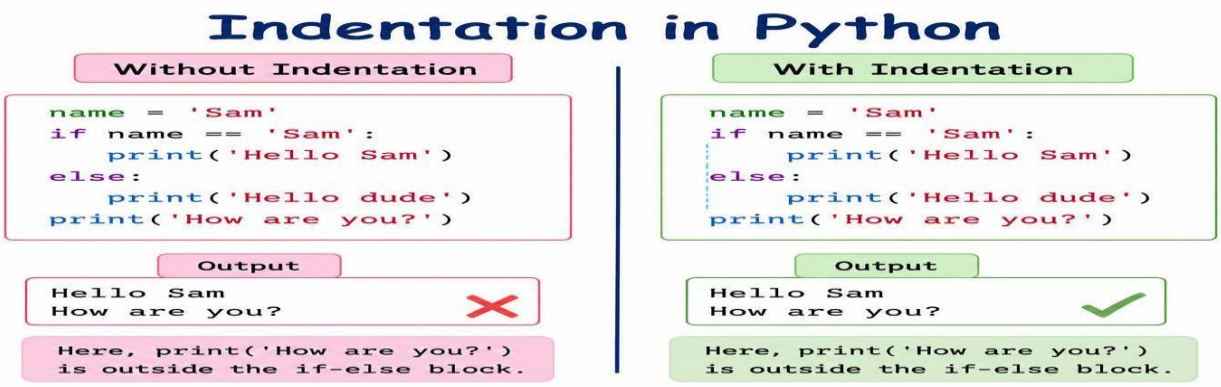
Some examples of valid expressions are given below.

(i) num – 20.4 (iii) 23/3 -5 * 7(14 -2)

(ii) 3.0 + 3.14 (iv) "Global"+"Citizen"

5 Indentation: Python uses indentation for block as well as for nested block structures. Leading whitespace (spaces and tabs) at the beginning of a statement is called indentation.

A block is a piece of Python program text that is executed as a unit.



PYTHON CHARACTER SET:

Python character set represents the set of valid characters that Python supports. It has the following character set

CHARACTERS IN PYTHON

TYPE	DESCRIPTION	EXAMPLES
Aa Letters	Alphabets used in Python. Uppercase and lowercase.	A – Z, a – z ABC
123 Digits	Numbers used in Python.	0 – 9 0 1 2 3 4 5 6 7 8 9
!@# Special Symbols	Symbols having special meaning in Python.	Space + – * / ** \ { } ( ) // = != == < , > . " " " ; : % ! & # <= >= @ _ (underscore) </>
Whitespace	Blank space or invisible characters used for formatting.	Blank Space, Escape characters(Newline \n, Tab \t , Carriage Return \r , Form feed \f)
... Other Characters	Python can process all ASCII (American Standard code for Information Interchange) and Unicode characters as part of data or literals.	Python supports all ASCII characters (0 – 127) and Unicode characters (e.g., é, ü, 你好, 😊) in strings and literals.

Note: Python is a case-sensitive language. 'A' and 'a' are different.

3.2.Keywords or Reserved Words:Each keyword in python has a specific meaning to the Python interpreter.keywords cannot be used as identifiers, variables & functions and Python has the 33 keywords. Except "False", "None", and "True" all the keywords are in lowercase.

Table 3.1 Python keywords

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

To get the updated keywords list type below program

```
import keyword
k = keyword.kwlist
for i in k:
    print(i, end= "    ")
```

```
>>> import keyword
>>> print(keyword.kwlist)
```

3.3.Identifiers: In Python programming language, identifiers are names used to identify a variable, function, or other entities in a program.

The 4 rules for naming an identifier in Python are as follows:

1. Name should begin with **uppercase** (A-Z) or **lowercase** (a-z) or **underscore** sign (_).
2. It can be of any length (it is preferred tokeep it short and meaningful)
3. An identifier **cannot** start with a digit & It should **not** be a keyword
4. We cannot use special symbols like !, @, #, $, %, etc.in identifiers

Valid Identifiers	Invalid Identifiers	Remarks
_marks	!marks	No Special Character
Myfile	My file	No Space
syed_rafiq	syed-rafiq	Hyphen given
name	1name	Can't start with digit
true	True	Reserve words can't use

Variables:A variable is an identifier whose value **can change** during program execution.In Python, we can use an assignment operator(=) to create new variables and assign specific values to them.

Ex: gender = 'M' , message = "Keep Smiling" , age=30

Programme:

```
gender = 'M'
message = "Keep Smiling"
age = 30
height = 5.8
result = True
print(gender, type(gender))
print(message, type(message))
print(age, type(age))
print(height, type(height))
print(result, type(result))
```

Multiple Assignments: Different ways of assignments are

1. Assigning same value to multiple variables

Ex: a = b = c = 18

2. Assigning multiple values to multiple variables

```
Ex1:    x, y, z = 10, 20, 30          # Means x=10, y=20, z=30
Ex2:    x,y = y,x                     # This makes x=20, y = 10
Ex3:    a, b, c = 5, 10, 7
        a, b, c = a+1, b+2, c-1
        print (a, b, c)               # a=6, b=12, c=6
```

Displaying type of variable: The type( ) can be used to display the data type of a variable.

```
>>> a=10                >>> a=20.5              >>> a="Python"   >>>a=True
>>> type(a)             >>> type(a)             >>> type(a)       >>>type(a)
<class 'int'>           <class 'float'>         <class 'str'>      >>><class 'bool'>
```

id of an object: The id of an object is the memory location of it. The function id( ) is used for this purpose Ex:

```
>>> x=10
>>> id(x)
8791136999472
```

Constant: A **constant** is a value that should **not change** during the execution of the program Eg:PI= 3.14; gravity=9.8

```
PI = 3.14
GRAVITY = 9.8
DAYS_IN_WEEK = 7
MONTHS_IN_YEAR = 12
MAX_MARKS = 100
PASS_MARKS = 35
SPEED_OF_LIGHT = 300000

print("PI =", PI)
print("GRAVITY =", GRAVITY)
print("DAYS_IN_WEEK =", DAYS_IN_WEEK)
print("MONTHS_IN_YEAR =", MONTHS_IN_YEAR)
print("MAX_MARKS =", MAX_MARKS)
print("PASS_MARKS =", PASS_MARKS)
print("SPEED_OF_LIGHT =", SPEED_OF_LIGHT)
```

Input() And Output() functions In Python:

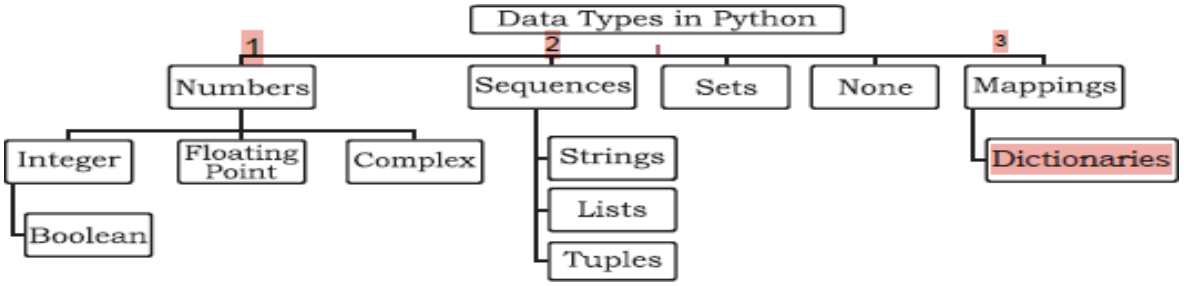
Input:-In Python, we have the input() function for taking values entered by input device such as a keyboard. The input() function prompts user to enter data.

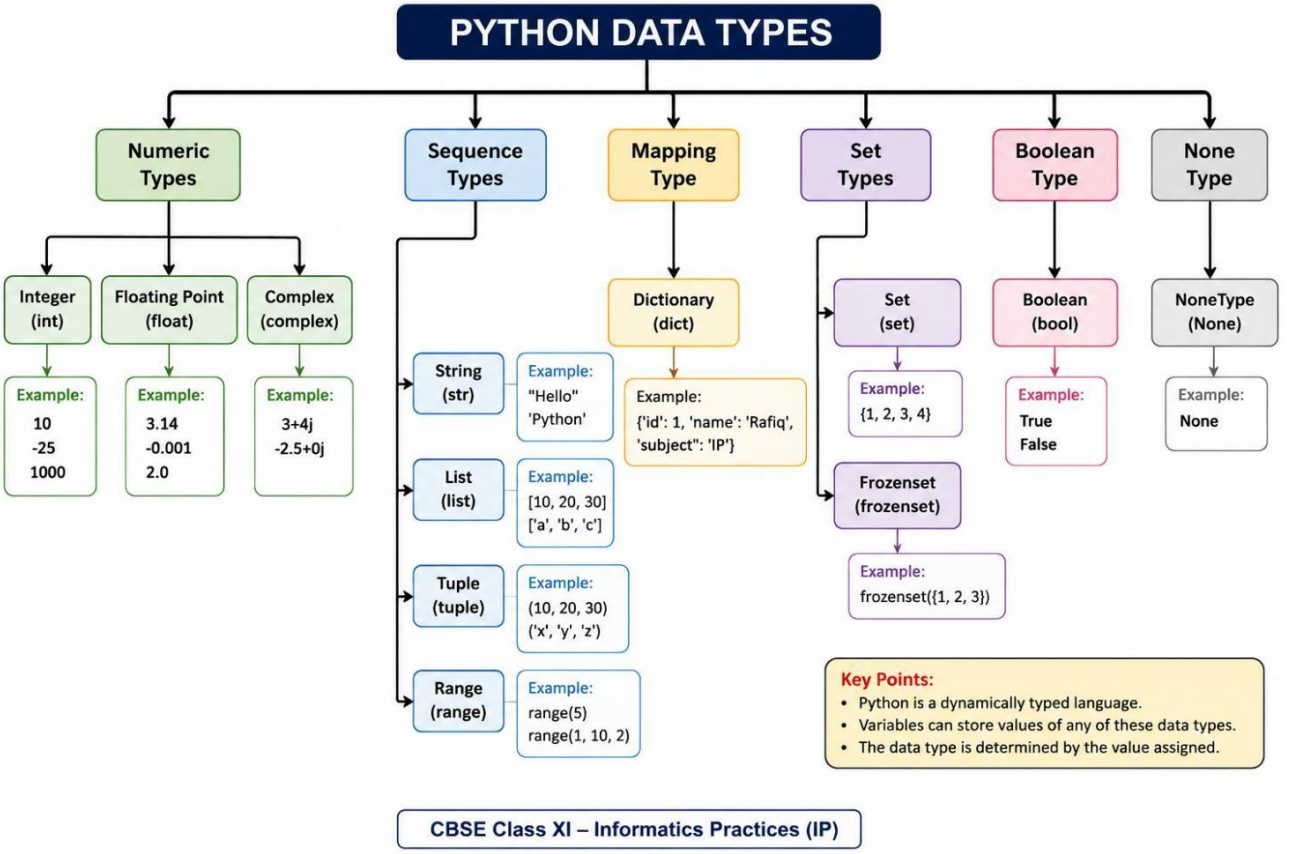
Syntax for input() is: variable = input([Prompt]) Eg:>>> name = input("Enter your name: ")

Print():Python uses the print() function to output data to standard output device — the screen. The function print() evaluates the expression before displaying it on the screen

Syntax for print() is: print(value) Eg:>>>print("Hello")

Data Types in Python:





1. Numeric Types:-Used to store numbers.

(a) **Integer (int)** :Stores whole numbers without decimal points. Ex: a=10,b=-20,c=20

(b) **Floating Point (float):-** Stores numbers with decimal points. Ex:-a=3.14,b=-0.001,c=2.0

(c) **Complex (complex):**Stores numbers having a real & imaginary part.Ex:- a = 3 + 4j, b = 2 + 6j

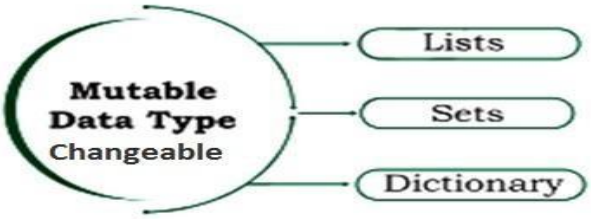
(d) **Boolean data type (bool):-** is a subtype of integer. It is a unique data type, consisting of two constants, True(0) and False(1).Ex:

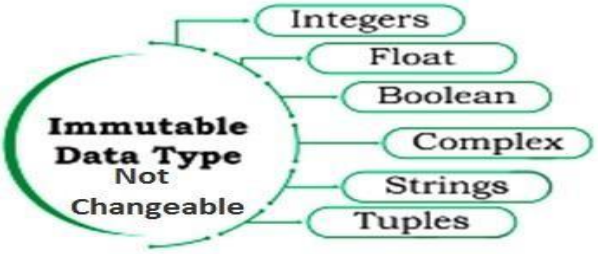
a=10 b=20 print(a>b) or print(a<b) True or False	a='Rafiq' b='Syed' print("Rafiq"=="Rafiq") True print("Rafiq"=="Syed") False	pwd="abc123" print(pwd=="abc123") True print(pwd!="abc123") False	today="Tuesday" print(today=="Sunday") False print(today=="Tuesday") True

MUTABLE AND IMMUTABLE DATA TYPES:

Mutable Types: The mutable data types are those that can **change their value** in place. Lists, Sets and Dictionaries and are mutable types. You can add, remove, or modify elements without creating a new object.

Immutable Types: The immutable data types are those that **cannot change their value** in place. Integers, Float, Booleans, Complex,Strings and Tuples are immutable types





(a)Sequences:- A **Sequence Data Type** stores multiple values in a specific order. Each element has a position called an **index**. Python provides **THREE** sequence data types like Strings, Lists, Tuples.

(a) Strings: String is a sequence of characters, Every character in a string has an **index ,**and Every character has two indexes **Forward Indexing**(0,1,2,3…) to and **Backward Indexing**(-1,-2,-3,…)

Index	0	1	2	3	4	5
String	P	Y	T	H	O	N
Index	–6	–5	–4	–3	–2	–1

Ex:

a="python" a[0] 'p' a[4] 'o' a[5] 'n'	a=”python” a[-1] 'n' a[-5] 'y' a[-6] 'p'

Note: A string is a sequence of characters enclosed in single quotes, double quotes, or triple quotes. All contents of a string are treated as characters.

(b) Tuples: Tuples are a sequence of values of any data type, and are indexed by integers.

tuples are immutable (we cannot modify). Tuples are enclosed in parentheses () separated by commas(,). Example: t1=(1,2.3,’jnv’)

- ( ) or **tuple()**----------> Empty Tuple
- (1,2,3) ----------> Tuple of Integers
- (1,2.4,3.5,9) ----------> Tuple of Numbers (int and float)
- (‘a’ , ‘b’ , ‘c’) ----------> Tuple of Characters
- (‘Nisha’ , ‘Neha’ , ‘Soniya’) ----------> Tuple of Strings
- (‘a’ , 1 , 3.5, ‘Python’) ----------> Tuple of mixed value types

Single Element tuple: To construct a tuple with one element add **comma (,)** after single element

Ex: t2=(1,)

Nested tuple: If a tuple contains an element , which is a tuple itself

Eg:t3=(1,2,(3,4,5),6,7), t4=((1,'eng',),(2,'ip'),(3,'acc'), (4,'bst'), (5,'eco'))

(c) **Sets:** A set is an unordered collection of unique elements, Duplicate values are not allowed in a set. Sets **not** allowed indexing and slicing, they are mutable (changeable). Sets are enclosed in curly brackets { }.

Example: s={1,2.3,'jnv'},
s1=set([1,2.3,'jnv']),
s2= {10, 20, 10, 30, 20}
:s3={10,True,'Rafiq',1.43,1}→Duplicate items not allowed Note:True is considered as 1

- Set()----------→Empty Set
- {1,2,3}----------→Set of Integers
- {1,2.4,3.5,9}----------→Set of Numbers(Int and Float)
- ('a','b''c'}----------→Set of Characters
- {'Python','AI','Data Science'}----------→Set of Strings
- {'a',1,4.3,'Pythpn'}----------→Set mixed value types

Adding Element in set

```
>>> s={10,20,30}
>>> print(s)
{10, 20, 30}
>>> s.add(40)
>>> s
{40, 10, 20, 30}
```

Removing Element in set

```
>>> s={10,20,30,40}
>>> print(s)
{10, 20, 30,40}
>>> s.remove(40)
>>> s
{10, 20, 30}
```

Empty set

```
a=set()
```

Update set

```
s.update([50,60,70])
```

d) **Lists:** List is a sequence of values of any type. Values in the list are called elements / items. They are mutable and indexed/ordered. List are enclosed in **square brackets []**.

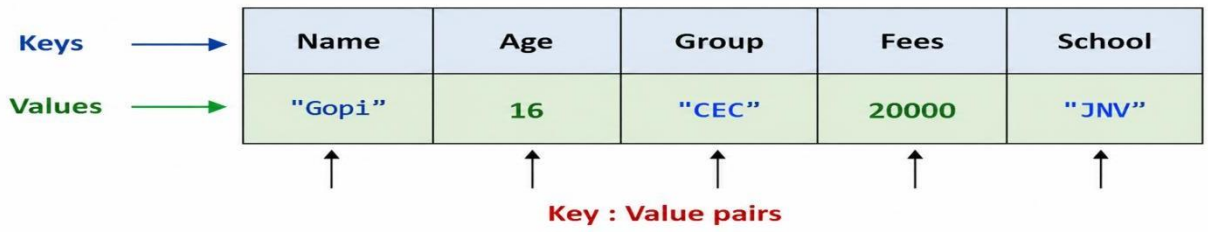
Example: l = [1, 2.3,'jnv']

Mapping:-Mapping is an unordered data type in Python. Currently, there is only one standard mapping data type in Python called Dictionary.

Dictionary: Dictionary is an unordered data type in Python holds data items in **key-value pairs**, They are mutable. Dictionaries are enclosed in curly brackets { }

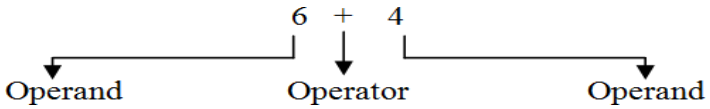
Ex: d={"Name" : "Gopi", "Age":16,"Group":"CEC","Fees":20000,"School":"JNV"}

```
d = {"Name" : "Gopi", "Age":16, "Group":"CEC", "Fees":20000, "School":"JNV"}
```

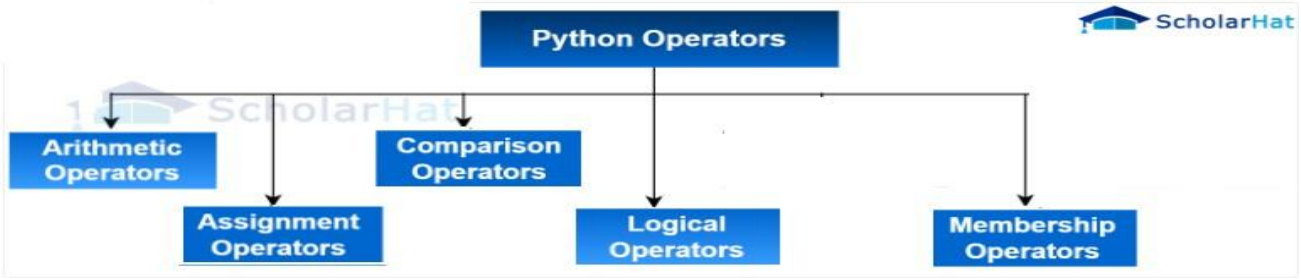


Operators: An operator is a symbol that is used in a program in respect of some operation. Each programming language will have its own set of operators.

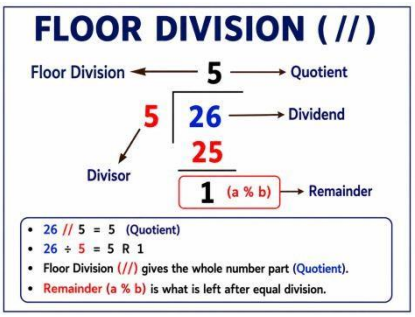
The constants or variables that participate in the operation are called operands



Unary Operator: If an operator takes only one operand Ex:5
Binary Operator: If an operator takes two operands Ex: 1+3=4
Python Operators: Python supports 5 kinds of operators.



1. Arithmetic Operators: Python supports arithmetic operators to perform the four basic arithmetic operations (+ - *), /→True division (In true division the result of dividing two integers is a float) , floor or integer division(In floor division the result of dividing two integers is a integer) value gives (//)→Quotient, modulus division(%)→Remainder, and exponentiation (**)→Power(^)




```
# Define two numbers
a = 15
b = 4
print(a + b)    # Addition
print(a - b)    # Sub
print(a * b)    # mul
print(a / b)    # div
print(a // b)   # floor div (It removes the decimal part)
print(a % b)    # The modulus operator gives the remainder after division.
print(a ** b)   # exponent
```

2. Assignment Operators: are used to assign or changes the value of the variable on its left.

There are 7 ASo's in Python

```
a = 10
b = 2
a += b;        print("a += b:", a)
a -= b;        print("a -= b:", a)
a *= b;        print("a *= b:", a)
a /= b;        print("a /= b:", a)
a %= b;        print("a %= b", a)
a //= b;       print("a //= b:", a)
a **= b;       print("a **= b:", a)
```

Step-by-step

Statement	Calculation	New Value of `a`
`a = 10`	Initial value	10
`a += b`	10 + 2	12
`a -= b`	12 - 2	10
`a *= b`	10 × 2	20
`a /= b`	20 ÷ 2	10.0
`a %= b`	10.0 % 2	0.0
`a //= b`	0.0 // 2	0.0
`a **= b`	0.0^2	0.0

3. Relatonal or Comparison Operators: are used to compare two values and return a Boolean result (True or False).

There are 6 RO's in Python

1.Greater than(>) 2.Greater Than or Equal To(>=), 3. Less Than(<),
4. Lesser Than or Equal To(<=),5. Equal To(==),6. Not Equal To(! =)

```
a = 10
b = 5

print(a > b)
print(a >= b)
print(a < b)
print(a <= b)
print(a == b)
print(a != b)
```



4. Logical Operators: Logical operators are used to **combine two or more conditions** and return either **True** or **False**. There are **three** logical operators supported by Python. These operators **(and, or, not)** are to be written in lower case only.

1. **and: and** operator is used by applying(.-->dot) between two variables, it is just like multiplication. **If both operands are True, then condition becomes True**

```
a=5
b=3
print(a>b and a<b)
False
print(a>b and b<a)
True
```

```
a=26
b=54
print(a<b and b>a)
True
print(a<b and b<a)
False
```

```
a=23
b=67
print(a<b and a>b)
False
print(b>a and a<b)
True
```

X	Y	X . Y (X AND Y)
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

2. **or : or** operator is used by applying(+-->plus) between two variables, it is just like addition. If any one of the operands is True, then condition becomes True

```
a=23
b=45
print(a<b or b<a)
True
```

```
a=23
b=45
print(b<a or a>b)
False
```

X	Y	X + Y (X OR Y)
0	0	0
0	1	1
1	0	1
1	1	1

x	y	x ory
False	False	False
False	True	True
True	False	True
True	True	True

3. **Not**: Used to reverse the logical state of its operand.It returns True if the conditional expression returns False, and vice-versa.

x	not x
0	1
1	0

x	notx
False	True
True	False

#WAPP by using LO operators

```
x=10
y=20
print(x==10 and x<y)
print(x==10 or x<y)
print(not x!=10)
```

```
x=10
y=20
print(x==y)
False
print(x!=y)
True
```

```
x=10
y=20
print(x==y)
False
print(x!=y)
True
print(not x!=y)
False
```

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Membership Operators : is used to check if a value is a member of the given sequence or not. There are two membership operators in python **in** and **not in**.

1. in Operator: Returns True if the variable or value is **found in** the specified sequence and False otherwise.

```
>>> a=[1,2,3]
>>> 2 in a
True
>>> 4 in a
False
```

2. Not in: Returns True if the variable/value is **not found in** the specified sequence and False otherwise

```
>>> a=[1,4,3]
>>> 10 not in a
True
>>> 1 not in a
False
```

Debugging: The process of identifying and removing errors is called debugging. Python supports **three** types of errors

1. Syntax Error 2.Logical Error 3.Runtime Error

1. Syntax Error: In Python, a syntax error occurs when the interpreter encounters code that doesn't conform to the language's grammar rules.

Ex: 1.print('Helo"), Ex: 2.print("Hello, 3.a=10

b=5

if(a<b)

print(x)

2. Logical Error:logical error/bug (called semantic error) does not stop execution but the program behaves incorrectly and produces undesired /wrong output

Ex:**if we wish to find the average of two numbers 10 and 12**

>>>int(10+12/2)

16

>>>

>>>int(10+12)/2

11.0

Ex: **Suppose if u want to add 2 numbers and type below code like this**

```
a=input("enter number1")
b=input("Enter number2")
c=a+b
print('add',c)
```

it will give output like this

1 2

To add 2 numbers use correct logical code

```
a=int(input("enter number1"))
b=int(input("Enter number2"))
c=a+b
print('Sum',c)
```

3. Runtime Error:A runtime error causes abnormal termination of program while it is executing. Runtime error is when the statement is correct syntactically, but the interpreter cannot execute it.

Ex:-For example, we have a statement having division operation in the program. By mistake, if the denominator value is zero then it will give a runtime error like "division by zero".

Ex1.:13/0 (ZeroDivisionError:)

Ex:2

>>>str = "Hello"

>>> print(str + 5) (TypeError)

Ex 3:int('bc') (Value Error)

Ex 4: c=[1,2,3,4]

print(c[4])(Index Error)

Precedence of Operators: Operator precedence is the order in which Python evaluates operators in an expression. Operators with **higher precedence** are evaluated **before** operators with lower precedence. If operators have the **same precedence**, their **associativity** decides the evaluation order

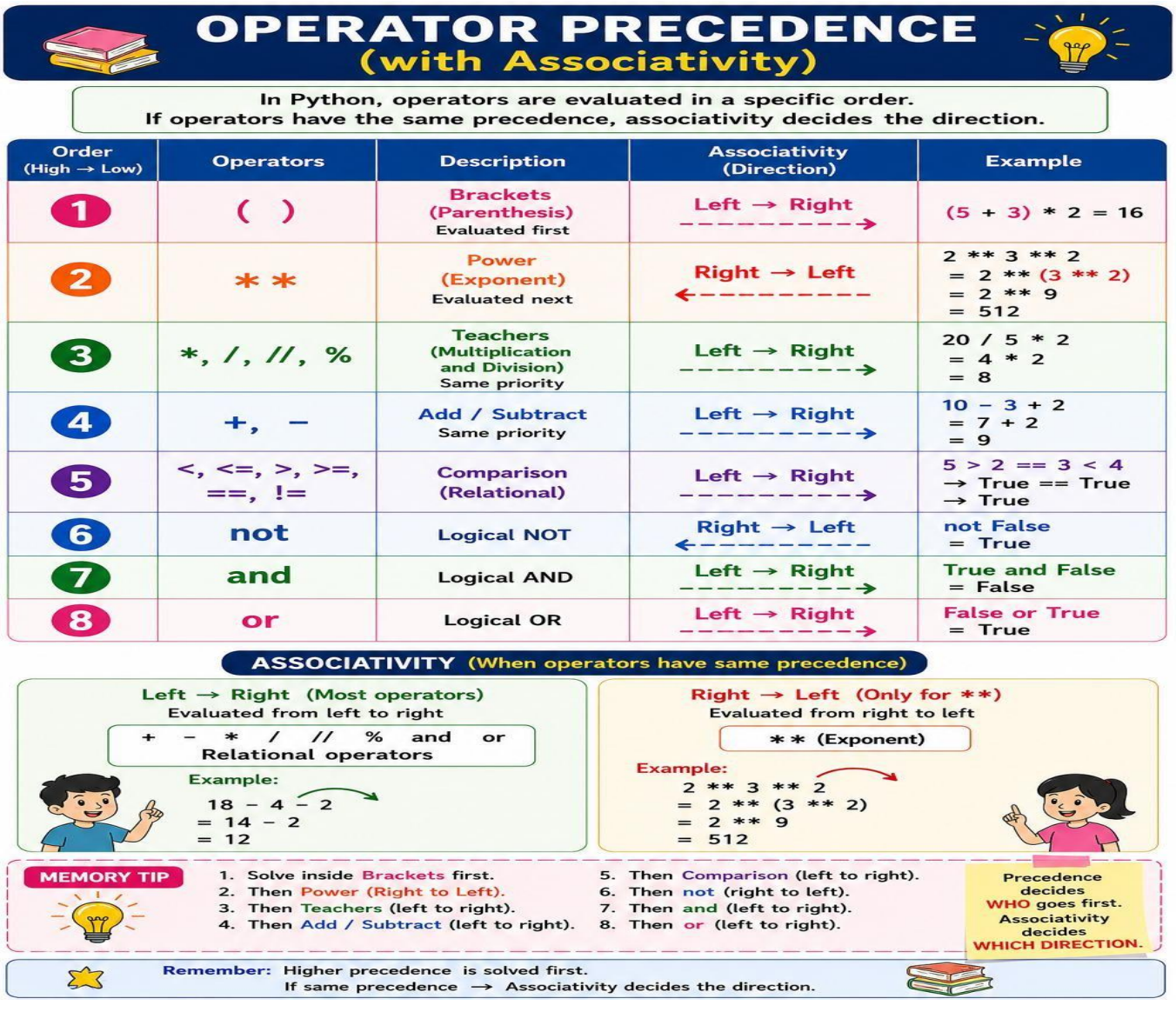
Operator Associativity: Almost all the operators have **left to right** associativity except **exponent operator (**)** which has **right to left** associativity.

In Python, **PEMDAS** is the rule that defines the order of operations when evaluating mathematical expressions.

PEMDAS stands for:

- **P → Parentheses** () – operations inside parentheses are done first
- **E → Exponents** ** – powers and roots **(Right to left)**
- **M → Multiplication** *
- **D → Division** / // %
- **A → Addition** +
- **S → Subtraction** -

□ Multiplication(*), Division(/), Floor Division(//), and Modulus(%) have the **same precedence** and are evaluated from **left to right**. Similarly, Addition and Subtraction are also evaluate from **left to right**.



Precedence	Operator Category	Operators
1 (Highest)	Arithmetic	`**`, `*`, `/`, `//`, `%`, `+`, `-`
2	Comparison (Relational)	`<`, `<=`, `>`, `>=`, `==`, `!=`
3	Logical	`not` → `and` → `or`
4 (Lowest)	Assignment	`=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`

Example 1 : print(10 + 2 * 3) [# Multiplication first → 10 + (2*3) = 16]
Example 2:- print((10 + 2) * 3) [# Parentheses first → (12) * 3 = 36]
Example 3 print(2 ** 3 * 2) [# Exponent first → (2**3) * 2 = 8 * 2 = 16]
Example 4 print(20 / 5 * 2) Division and Multiplication same precedence → (20/5) * 2 = 4 * 2 = 8.0
Example 5 print(15 - 3 + 2)
Subtraction and Addition left-to-right → (15-3)+2 = 12+2 = 14

Ex:6 2**2**3 Step 1:2**3=8 Step 2:2**8=256	Ex:7 14/2%2 Step 1:14/2=7.0 Step 2:7.0%2=1.0	Ex:8 20 + 30 * 40 Step 1:30*40=1200 Step 2:20+1200=1220
Ex:9 (20 + 30) * 40 Step 1:50 Step 2:50*40=2000	Ex:10 **15.0 / 4.0 + (8 + 3.0)** Step 1: 15.0 / 4.0 + (8.0 + 3.0) Step 2:15.0/4.0+11.0 Step 3:3.75 Step 4:14.75	Ex:11 **print(4.00/(2.0+2.0))** Step 1: 2.0 + 2.0 = 4.0 Step 2: 4.00 / 4.0 = 1.0
Ex 12: **23/3 -5 * 7*(14 -2)** Step1:23/3-5*7*12 Step 2:23/3-5*84 Step 3:23/3-420 Step 4:7.6-420 -412	Ex13 Step 1:**2+9*((3*12)-8)/10** Step 2:2+9*(36-8)/10 Step 3:2+9*28/10 Step 4:2+252/10 Step 5:2+25.2 Step 6:27.2	Ex:**14(7 == 7 or 3 > 8)** True or False True
Ex:15 **(5 == 5 and 6 == 7 or 8 > 4)** True and False or True False or True True	Ex:16 **10 + 5 > 12 and 8 - 3 == 5** 15>12 and 5==5 True and True True	Ex:17 **20 // 4 == 5 or 15 % 4 > 3** 5==5 or 3>3 True or False True
Ex:18 **not (12 // 3 == 4) and 8 > 5** not (4==4) and True not True and True False and True False	**9 > 5 and 12 % 5 == 2 or not 16 // 4 == 5** 9 > 5 and 12 % 5 == 2 or not 4 == 5 9 > 5 and 12 % 5 == 2 or not 4 == 5 9 > 5 and 2 == 2 or not 4 == 5 True and True or not False True and True or True True or True True	Ex20: **8 + 2 > 5 * 2 and 18 % 5 == 3** 8+2>10 and 18%5 == 3 8+2>10 and 3 == 3 10>10 and True False and True False

Data Type Conversion in Python: means changing the data type of a value from one type to another. Python supports two types of data type conversion.

a.Implicit Type Conversion b.Explicit Type Conversion(Type Casting)

1. **Implicit Type Conversion** (Automatic Conversion) :Done automatically by Python.

Integer+float	Integer+Boolean	Boolean*Float

a = 10 # int
b = 2.5 # float
c = a + b
print(c)
print(type(c))
Output:
12.5
<class 'float'>
	a=True
	b=5
	c=a+b
	print(c)
	print(type(c))
		a=False
		b=8.5
		c=a*b
		print(c)
		print(type(c))

2. **Explicit Type Conversion** (**Type Casting**):Done manually by the programmer. Uses built-in functions.

Function	Converts To	Example
int()	Integer	int(5.9) → 5
float()	Float	float(10) → 10.0
str()	String	str(25) → "25"
bool()	Boolean	bool(0) → False

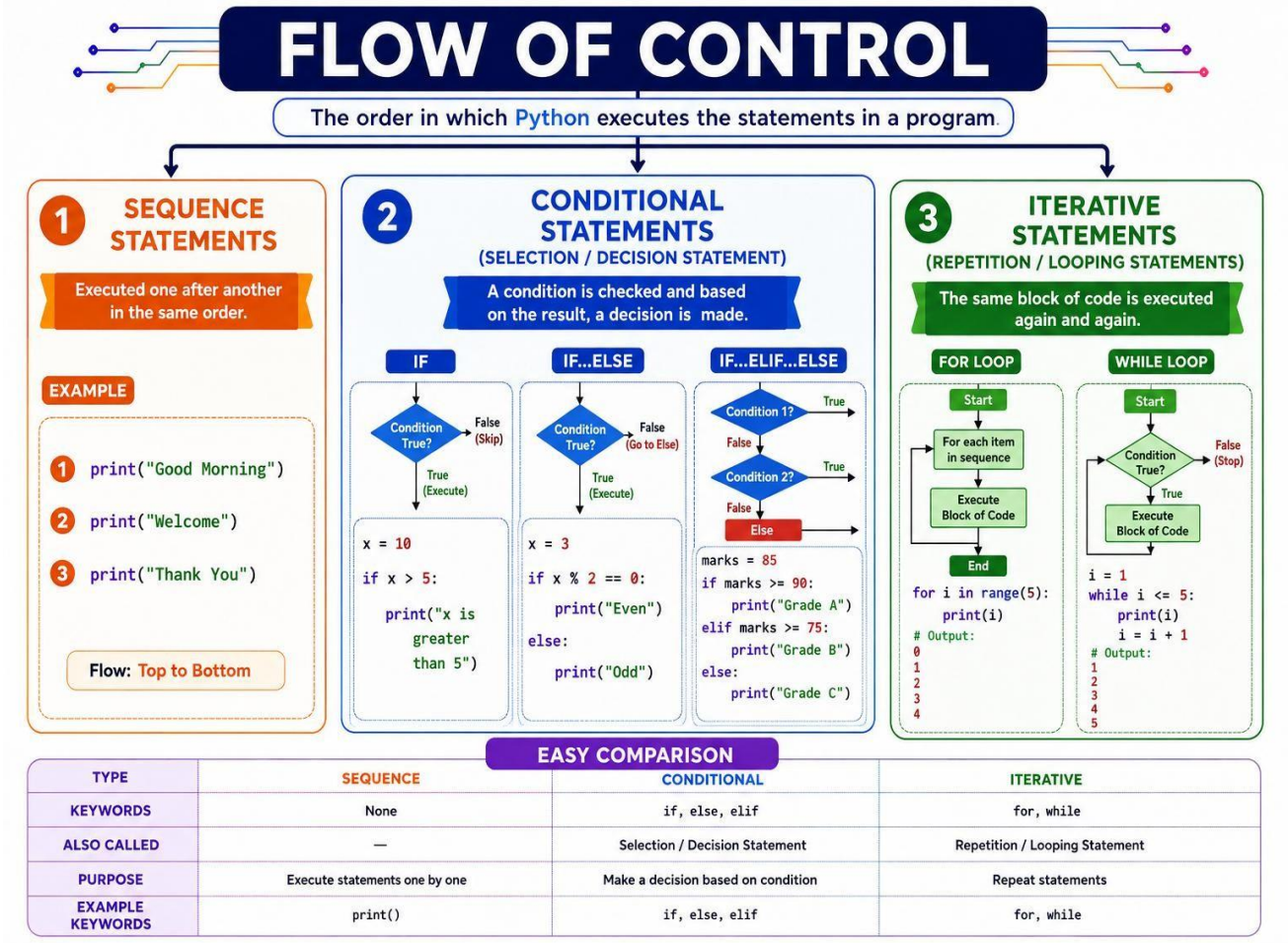
Integer to Float x=25 print(float(x))	**Integer to String** x=100 print(str(x)) or x = 100 y = str(x) print(y) print(type(y))	**Integer to Boolean** **print(bool(0))** **print(bool(25))** **Output:** **False** **True**
Float to Integer x=9.8 print(int(x))	**Float to String** a=45.6 b=str(a) print(b) print(type(b))	**Float to Boolean** a=5.7 b=bool(a) print(b) print(type(b))
String to Integer a="250" b=int(a) print(b) print(type(b))	**String to Float** x="15.75" print(float(x))	**String to Boolean** a="Python" b=bool(a) print(b) print(type(b))
Boolean to Integer a=True b=int(a) print(b) print(type(b))	**Boolean to Float** a=True b=float(a) print(b) print(type(b))	**Boolean to String** a=True b=str(a) print(b) print(type(b))

Important Clarification
Students often confuse **the Boolean value True** with **the integer 1**.
True is a Boolean value.
1, 2, 25, and -5 are integers.
When you use bool(), Python converts any **non-zero integer** to the Boolean value True.

A simple classroom statement
Boolean has only two values: True and False.
True is represented by 1.
False is represented by 0.
When converting numbers to Boolean, **0 becomes False and every non-zero number becomes True**.

Flow of Control: The order of execution of the statements in a program is known as flow of control. The flow of control can be implemented using control structures.

Python has three types of control structures—**Sequence, Selection, Iteration (repetition)**.



EASY COMPARISON

TYPE	SEQUENCE	CONDITIONAL	ITERATIVE
KEYWORDS	None	if, else, elif	for, while
ALSO CALLED	—	Selection / Decision Statement	Repetition / Looping Statement
PURPOSE	Execute statements one by one	Make a decision based on condition	Repeat statements
EXAMPLE KEYWORDS	print()	if, else, elif	for, while

1. **Sequence**: are a set of statements whose execution process happens in a sequence.

Ex:a=20
b=10
c=a-b
print("Subtraction is",c)

2. **Selection/Decision/Conditional**: The selection statement allows a program to test several conditions and execute instructions based on which condition is true.

Decision Making using – if elif else

For 1 condition : if
For 2 conditions : if else
For more than 2 conditions : if elif else

(a)If: if only has **one condition** to check.

WAPP using if condition(Only one condition)

```
age = int(input("Enter your age: "))
if age >= 18:
    print("Eligible to Vote")
```

```
password = input("Enter password: ")
if password == "12345":
    print("Login Successful")
```

(b)if-else: The if-else statement evaluates the condition and will execute the body of if if the test condition is True, but if the condition is False, then the body of else is executed.

WAPP using if..else condition(Two conditions)

```
marks = int(input("Enter marks: "))
if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

```
num = int(input("Enter a number: "))
if num >= 0:
    print("Positive Number")
else:
    print("Negative  Number")
```

```
age = int(input("Enter your age: "))
if age >= 18:
    print("Eligible to Vote")
else:
    print("Not Eligible to Vote")
```

```
password = input("Enter password: ")
if password == "12345":
    print("Login Successful")
else:
    print("Wrong Password")
```

```
a = input("Enter Result (Pass/Fail): ")

if a == "Pass":
    print("Congratulations!")
else:
    print("Better Luck Next Time")
```

(c) if-elif-else: if-elif-else statement is used to conditionally execute a statement or a block of statements.

WAPP using if..else condition(More than two conditions)

```
num = int(input("Enter a number: "))
if num > 0:
    print("Positive Number")
elif num < 0:
    print("Negative Number")
else:
    print("Zero")
```

```
marks = int(input("Enter marks: "))
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

```
salary = int(input("Enter salary: "))

if salary >= 100000:
    print("High Salary")
elif salary >= 50000:
    print("Medium Salary")
else:
    print("Low Salary")
```

```
marks = int(input("Enter marks: "))

if marks >= 75:
    print("Distinction")
elif marks >= 35:
    print("Pass")
else:
    print("Fail")
```

```
signal = input("Enter signal colour: ")

if signal == "Red":
    print("Stop")
elif signal == "Yellow":
    print("Ready")
else:
    print("Go")
```

```
per = float(input("Enter percentage: "))

if per >= 60:
    print("First Division")
elif per >= 45:
    print("Second Division")
else:
    print("Third Division")
```

3. **Repitition/Iteration/Looping statement:** is used to repeat a group(block) of programming instructions. Sometimes we need to repeat certain things for a particular number of times

In Python, we generally have two loops/repetitive statements:

- **for loop**
- **while loop**



(A) **for loop:** The for statement is used to iterate over a range of values or a sequence. The loop is executed for each item in the range

Syntax: for <control-variable> in <sequence/items in range>:
<statements inside body of the loop>

WAPP by using for condition

Print "Hello" 5 times
for i in range(5):
print("Hello") or
a=[1,2,3,4,5,6]
for i in a:
print("Hello")
	Print your name 10 times
	for i in range(10):
	print("Rafiq")
		Print numbers from 1 to 10
		for a in [1,2,3,4,5]: #steps
		print(a)
		for a in range(1, 11): #lift
		print(a)

Print no's from 10 to 1
for i in range(10, 0, -1):
print(i) or
for i in range(10,-1,-1):
print(i)
	Print square numbers from 1 to 10
	for i in range(1, 11):
	print(i ** 2)
		Print all characters of a string
		name = input("Enter your name: ")
		for ch in name:
		print(ch)

Print multiplication table of any number

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Print a star pattern

```
for i in range(1, 6):
    print("*" * i)
```

Print list values

```
for a in [1,2,3,4]:
     print(a)
```

To Print a square number

```
for a in (1,4,3):
    print(a)
    print(a*a)
```

Print even or numbers in the given sequence

```
a= [1,2,3,4,5,6,7,8,9,10]
for num in a:
    if (num % 2) == 0:
       print(num,'is an even Number')
    else:
      print(num,'is an odd Number')
```

Write a Python program to ask the user to enter the correct password 3 times using a for loop.

```
password = "1234"
for i in range(3):
    p = input("Enter Password: ")
    if p == password:
        print("Login Successful")
    else:
        print("Login Failed")
```

(B) **while loop:** The while statement executes a block of code repeatedly as long as the control condition of the loop is true. The control condition of the while loop is executed before any statement inside the loop is executed

WAPP by using WHILE condition

Print "Hello" 5 times

```
count = 1
while count <= 5:
    print("Hello")
    count = count + 1
```

Print your name 10 times

```
count = 1
while count <= 10:
    print("Syed Rafiq")
    count = count + 1
```

Print numbers from 1 to 10

```
count = 1
while count <= 10:
    print(count)
    count = count + 1
```

Print no's from 10 to 1

```
count = 10
while count >= 1:
    print(count)
    count = count - 1
```

Print square numbers from 1 to 10

```
num = 1
while num <= 10:
    print(num * num)
    num = num + 1
```

Print all characters of a string

```
name = "Python"
i = 0
while i < len(name):
    print(name[i])
    i = i + 1
```

Print multiplication table of any number

```
num = int(input("Enter a number: "))
i = 1
while i <= 10:
    print(num, "x", i, "=", num * i)
    i = i + 1
```

Print a star pattern

```
rows = 5
i = 1
while i <= rows:
    print("*" * i)
    i = i + 1
```

Print list values

```
a = [10, 20, 30, 40, 50]
i = 0
while i < =4:
        print(a[i])
    i = i + 1
```

To Print a square number

```
a=int(input("Enter a number: "))
i = 1
while i <= 1:
    print("Square=", a * a)
    i = i + 1
```

Print even or odd numbers in the given sequence

```
a = [1,2,3,4,5,6,7,8,9,10]
i = 0
while i < len(a):
    if a[i] % 2 == 0:
        print(a[i], "Even")
    else:
        print(a[i], "Odd")
    i = i + 1
```

Write a Python program to ask the user to enter the correct password 3 times using a while loop.

```
password = "1234"
i = 1
p = ""
while i <= 3 and p != password:
    p = input("Enter Password: ")
    if p == password:
        print("Login Successful")
    else:
        print("Login Failed")
    i = i + 1
```

The range() Function: is a built-in function in Python. It is used to create a list containing a sequence of integers from the given start value upto stop value.

Syntax:range(start, stop, step)

- **start** → Starting value (included)
- **stop** → Ending value (excluded)
- **step** → Difference between two numbers

The function range() is often used in for loops for generating a sequence of numbers

for i in range(5): print(i)	for i in range(1, 6): print(i)	for i in range(10, 15): print(i)
for i in range(-3, 4): print(i)	for i in range(2, 11, 2): print(i)	for i in range(5, 31, 5): print(i)
for i in range(1, 10, 2): print(i)	for i in range(10, 101, 10): print(i)	

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
#start value is given as 2
>>>  list(range(2, 10))
[2, 3, 4, 5, 6, 7, 8, 9]
#step value is 5 and start value is 0
>>>  list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
```

Ex2:

```
# Program to print numbers from 1 to 10 and indicate even or odd
for num in range(1, 11):  # 1 to 10
    if num % 2 == 0:
        print(num, "is an even number")
    else:
        print(num, "is an odd number")
```

Sno	For	While
1	Used when we **already know how many times** to repeat.	Used when we **do not know how many times** to repeat.
2	Runs a **fixed number of times**	
3		
4		
5		
6		
7		
8		
9		
10		